université de BORDEAUX

Automatic tuning of the IO500 benchmark parameters with Bayesian Optimization

M2 Computer Science
Internship Report

Inria Centre at the University of Bordeaux

Supervisors:

September 8, 2026

Abstract

Keywords: HPC, parallel I/O, IO500, auto-tuning, Bayesian Optimization, benchmarking, reproducibility.

Contents

Abstract **ii**

1. Introduction **1**
1.1. Objectives of the internship . . . 3
1.2. Contributions . . . 3
1.3. Organisation of the report . . . 3

2. Context of the internship **3**
2.1. Inria and the Inria Centre at the University of Bordeaux . . . 4
2.2. The host team: TADaaM . . . 4
2.3. Scientific and technical environment . . . 5
2.4. Objectives and specifications . . . 5
2.5. Organisation of the work . . . 6

3. Background **7**
3.1. IO500 . . . 7
3.1.1. Benchmark structure . . . 7
3.1.2. Scoring and ranking . . . 8
3.1.3. Execution rules and stonewall . . . 9
3.1.4. Configuration and tuning . . . 9
3.2. IOPS . . . 10
3.2.1. IOPS workflow . . . 10
3.2.2. Parameter configuration . . . 11
3.2.3. Search strategies . . . 12
3.2.4. Result management and caching . . . 12
3.3. Bayesian Optimization . . . 13

4. Methodology **14**
4.1. Parameter space . . . 14
4.2. Exhaustive search . . . 16
4.3. Ground-truth construction . . . 20
4.4. Search strategies . . . 21
4.4.1. Random Search . . . 21
4.4.2. Bayesian Optimization . . . 22

5. Results **24**
5.1. Characterisation of the response surface . . . 24
5.2. Which Bayesian Optimization setup wins? . . . 25
5.3. Why random search is not enough . . . 25
5.4. Automated search versus a human specialist . . . 25
5.5. Validation with valid IO500 runs . . . 25

6. Conclusion and perspectives **25**
6.1. Summary . . . 25
6.2. Perspectives . . . 26

Bibliography **27**

1. Introduction

High Performance Computing (HPC) systems are used to run large-scale scientific and engineering applications. The applications use many compute resources in parallel and often process large amounts of data. Therefore, an application's performance does not only depend on the computing power of the system, but also on the performance of its storage infrastructure. In many HPC workloads, I/O operations become the bottleneck that limits overall application performance.

Parallel file systems (PFS) such as Lustre [12], BeeGFS [17] and GPFS [11] are commonly used to provide high-performance storage in HPC environments. In a PFS, the storage capacity is exposed as a set of logical units called Object Storage Targets (OSTs). When a file is written, it is split into fixed-size chunks that are sent in parallel to several targets, so that the bandwidth of many storage devices is used at once. This process is known as striping and is illustrated in Figure 1. How many targets a file is spread over, called the stripe count, and how large each chunk is, called the stripe size, are configuration choices rather than fixed properties of the system. Therefore, a PFS exposes configuration parameters that can affect I/O performance. Beyond striping, performance can also depend on the transfer size, the number of processes, or the placement of processes on compute nodes. The best values for these parameters are not necessarily the same for every machine: a configuration that performs well on one system may perform poorly on another one.

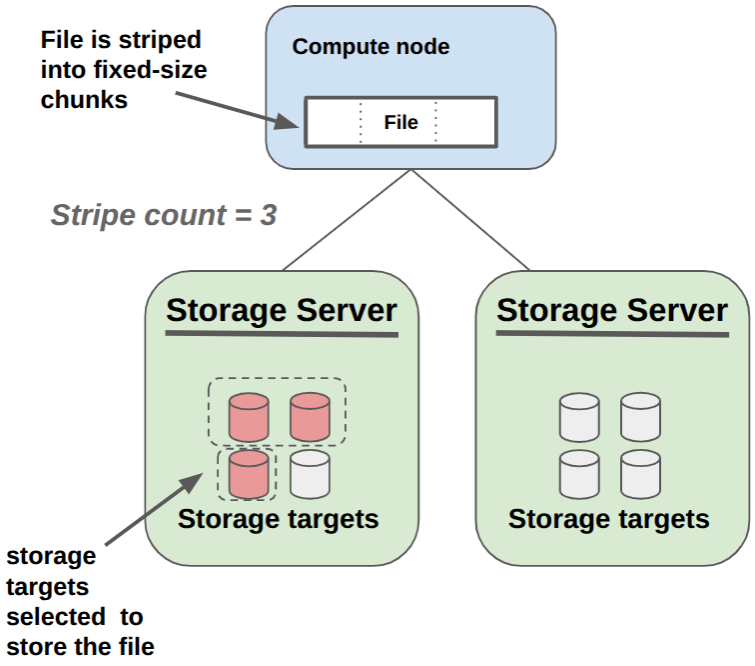


Figure 1: Striping in a parallel file system. A file created on a compute node is split into fixed-size chunks, which are written in parallel to storage targets hosted by the storage servers. Here the stripe count is three, so the file is spread over three of the eight available targets.

To evaluate the performance of these systems, we rely on benchmarks. A benchmark is a program designed to stress a specific part of a system under controlled and reproducible conditions, so that

the measurements it produces can be compared across machines and configurations [2]. For I/O, two of the most widely used benchmarks are IOR [14], which measures the bandwidth reached by large parallel read and write operations, and mdtest [15], which measures the rate at which metadata operations such as creating, opening and removing files are performed. A different approach is taken by mini-applications, or miniapps [1], which reproduce the access pattern of a real scientific application in a reduced and self-contained form, and therefore measure performance under a workload closer to production use. Each of these benchmarks characterises one aspect of the system, and none of them alone describes the storage system as a whole.

Thus, the variety of parameters and access patterns makes the evaluation of I/O performance difficult: a benchmark must not only measure the performance of the storage system, but also provide a meaningful way to compare different systems and configurations. For this reason, the HPC community maintains rankings in which every system runs the same benchmarks under a common set of rules, so that the resulting scores can be compared and ordered. The best known of them is the TOP500 [21], which ranks HPC platforms by their computational performance. In the I/O world, the community created the IO500 benchmark suite [9]. IO500 is widely used to evaluate and rank the I/O performance of HPC storage systems, and it combines several workloads, including bandwidth-oriented and metadata-oriented operations, into a single score, which is used to produce the IO500 ranking.

Because IO500 is a ranking, there is a direct incentive for HPC sites to obtain the best possible score. For vendors and system integrators, a high position is a commercial argument, used in marketing material and in procurement discussions. For computing centres, it is a public and comparable demonstration that the machine they operate delivers the capability it was built for, which matters both for the visibility of the centre and for justifying the investment it represents. However, the reported performance of a system does not only depend on its hardware, but also on how the system is configured and how the benchmark is executed. Finding a good configuration is therefore essential to obtain the highest possible score.

However, there is a large number of possible configurations, which makes the tuning of IO500 difficult: a configuration can involve several parameters, and each parameter can have multiple possible values. The combination of these parameters creates a potentially large search space, and finding a good configuration therefore requires evaluating many possible combinations.

Another important difficulty is the cost of an IO500 evaluation. According to the IO500 submission rules [10], each write and create phase must run for at least 300 s. A run that finishes the write phase earlier is considered invalid. This requirement makes a valid IO500 measurement relatively expensive in terms of cluster time: on the target platform, one valid measurement takes approximately 30 min of exclusive cluster time. When several configurations and repetitions are considered, the total cost becomes very large. For instance, in the parameter space considered in this work, approximately 7,840 configurations have to be evaluated. Measuring all of them with three repetitions would require approximately 12 000 h of wall-clock time, which corresponds to more than one year of computation on the target machine. Therefore, An exhaustive search is not practical.

This leads to the main research question of this internship:

> *Can a smarter search method find a near-optimal IO500 configuration using far fewer evaluations, and which search method is most effective for this problem?*

1.1. Objectives of the internship

The objective of this internship is to tune IO500 with a guided search instead of an exhaustive one, and to determine which search method should be used in practice. Bayesian Optimization is the main candidate, but it is a family of methods rather than a single one, so a large part of the work consists in comparing its variants, and comparing them against random and exhaustive search.

The work is carried out with IOPS [13], a benchmark orchestration framework developed in the team. IOPS describes a parametric study in a single YAML file and takes care of generating the runs, submitting them to the cluster scheduler, parsing their output and caching the results. Two of its features matter here: the search strategy is a configuration option, so exhaustive, random and Bayesian searches can be compared without changing anything else; and a study can be replayed from the cache, which is what makes the offline comparison possible.

1.2. Contributions

The main contributions of this work are the following.

- *A cheap ground-truth methodology* to evaluate search strategies for tuning IO500, based on a set of invalid scores that are much cheaper to generate but still allow the strategies to be compared.
- *A systematic comparison of Bayesian Optimization variants*, which identifies the setup that performs best and shows which of its parameters actually matter.
- *A validation campaign* on the cluster, checking that the configurations selected on the cheap proxy remain the best ones under the official IO500 rules.

1.3. Organisation of the report

The remainder of this report is organised as follows. The next section presents the host laboratory and the environment in which the internship took place. The background section then introduces what is needed to read the rest of the report: the IO500 benchmark and its rules, the parameters exposed by parallel file systems, and the principles of Bayesian Optimization. Related work on I/O auto-tuning is reviewed next, followed by a description of the tools, the platform and the experimental setup. The methodology section explains how the ground truth was built and how the search strategies were compared, and the results section reports and analyses the measurements. The report closes with a discussion of the limitations of the study and a conclusion.

2. Context of the internship

This section describes the context in which the internship took place. We first present Inria and the Inria Centre at the University of Bordeaux, followed by TADaaM, the team that hosted the internship. We then describe the scientific and technical environment in which the work was carried out, the objectives that were defined for the internship, and the way the work was organised.

2.1. Inria and the Inria Centre at the University of Bordeaux

Inria is the French national research institute for digital science and technology. Its research activities cover a wide range of topics in computer science, applied mathematics, and digital technologies. Research is organised into project teams, which bring together researchers, engineers, PhD students, and other scientific staff around common research topics.

The internship was carried out at the Inria Centre at the University of Bordeaux. The centre is located on the Talence and currently has 22 project teams (20 teams located in Talence and two teams in Pau). The centre brings together more than 470 researchers, engineers, technical staff, and other members of the research and innovation community. These groups are developed in partnership with institutions such as the University of Bordeaux, Bordeaux INP, CNRS, Inserm, INRAE, and other academic and industrial partners [3].

The research activities of the centre cover several areas of digital science. These include high-performance computing and Big Data, machine learning, and digital health. This diversity reflects the different research problems addressed within the centre, ranging from fundamental computer science to applications in science, engineering, and health. For the work presented in this report, the high-performance computing research activities are particularly relevant.

In the centre, the STORM, TADaaM, and CONCACE teams conduct research on high-performance computing, systems, optimisation, and the execution of parallel applications [5]. These teams contribute to a local research environment in which computing performance, system software, and large-scale data management are studied using both theoretical approaches and experiments on real HPC platforms.

2.2. The host team: TADaaM

The internship was carried out within TADaaM [6]. The team works on data management for high-performance computing systems, with a particular interest in the interaction between applications, computing resources, networks, and storage systems. It studies how data can be managed efficiently at system scale and how the characteristics of the underlying infrastructure affect application performance.

The internship is part of the team's activities related to storage performance characterisation and I/O benchmarking. In particular, the work contributes to the study of how HPC storage systems can be configured and evaluated using the IO500 benchmark suite[7]. The internship also makes use of IOPS[13], a benchmark orchestration and optimisation framework developed within the team. The framework provides the tools needed to automate benchmark executions, explore different parameter configurations, and collect and analyse their results.

The internship was supervised by Luan Teylo, a researcher working within the TADaaM team. The work was also connected to a broader collaboration with international researchers that collaborate with the IO500 consortium. . These interactions provided feedback on the benchmark configuration, experimental methodology, and interpretation of the results.

2.3. Scientific and technical environment

The experiments conducted during the internship were carried out on PlaFRIM, the experimental computing platform used by the Inria research community in Bordeaux. PlaFRIM provides access to several types of computing nodes and is designed to support experiments on high-performance computing systems and new hardware technologies [4].

The experiments presented in this report were conducted on the `bora` nodes, which consist of 44 nodes, from `bora001` to `bora044`. Each node is equipped with two 18-core Intel Xeon Gold 6240 processors, providing 36 physical CPU cores per node, and 192 GB of memory. The nodes are connected through a 100 Gbit/s Omni-Path network, as well as a 10 Gbit/s Ethernet network.

The use of a dedicated and homogeneous set of nodes is important for the experiments carried out in this work. Since the objective is to compare different IO500 configurations, changes in the measured performance should primarily result from the parameters being studied rather than from differences between compute nodes. [4].

The experiments were managed using SLURM, the workload manager used on PlaFRIM. SLURM was used to request computing resources and to execute benchmark experiments on the selected nodes. Depending on the experiment, jobs could therefore be allocated with a specified number of nodes and resources. This was particularly important for the internship because the number of compute nodes and processes is itself part of the parameter space explored when tuning IO500.

The storage system used for the experiments is BeeGFS, a parallel file system designed to provide high-performance access to shared storage. The benchmark data and results were stored on the BeeGFS file system provided by PlaFRIM. The parallel file system is accessed through a 100 Gbit/s Omni-Path network. In addition to BeeGFS, the `bora` nodes provide a local 1 TB disk mounted under `/tmp`. The experiments described in this report, however, focus on the performance of the parallel storage system rather than the local disks [4].

The software environment consisted of Linux together with the tools required to build and execute the benchmarks. Python was used for the development and execution of the scripts associated with the experiments. OpenMPI was used as the MPI implementation required by the parallel benchmark workloads. The main research tools were IO500, used to evaluate the I/O performance of the system, and IOPS, used to automate the execution and exploration of different benchmark configurations.

The development process was split between the PlaFRIM environment and a local development machine. The source code, scripts, and experimental configurations were managed using Git and GitLab. This allowed the development work and experimental results to be kept under version control while the computationally intensive benchmark executions were performed on the cluster.

2.4. Objectives and specifications

The main objective of the internship was to tune the IO500 benchmark on the PlaFRIM `bora` cluster and determine whether automated search strategies could find good configurations with fewer evaluations than an exhaustive search.

From the beginning, the main elements of the study were fixed: IO500 was used as the benchmark, PlaFRIM `bora` as the target platform, and IOPS as the framework for running the experiments.

Bayesian Optimization (BO) was selected as the main search strategy, with random search and exhaustive search used for comparison.

The first stage of the internship focused on the exhaustive search. The objective was to explore the parameter space and obtain measurements that could later be used to compare the different search strategies. Each configuration was also evaluated several times to account for the variability of HPC measurements.

The study then focused on comparing the different search strategies under a limited number of evaluations. In particular, the objective was to determine whether BO could find configurations close to the best configurations found by exhaustive search while requiring significantly fewer evaluations.

As the internship progressed, the scope was extended to the development of IOPS. A plugin for running IO500 experiments was added to IOPS, together with an example YAML configuration describing the benchmark and the parameters explored during the study. This made it possible to run the IO500 experiments directly through IOPS and provided a reproducible configuration for the experiments.

The IOPS documentation was also updated to describe the new functionality and its usage.

The search component of IOPS was also extended with an implementation of the Tree-structured Parzen Estimator (TPE). At this stage, the method had been implemented but had not yet been evaluated on the IO500 experiments. Other optimisation methods, such as Iterated Racing (irace) and Covariance Matrix Adaptation Evolution Strategy (CMA-ES), were also considered as possible extensions.

These additional tasks emerged as the project evolved and extended the initial scope of the internship.

2.5. Organisation of the work

The internship took place from 14 April to 16 September. The work was organised progressively, starting with the study of the existing tools and the experimental environment, followed by the experimental campaigns and the development work.

The work was supervised by Luan Teylo through a weekly meeting of approximately 30 minutes. These meetings were used to review the progress of the work, discuss experimental results, and define the next tasks. Notes and follow-up tasks were recorded using the Inria Notes platform. Mattermost was also used for communication.

The work was managed using Git and GitLab. A personal repository was used to keep track of the work carried out during the internship, while the IOPS team repository was used for contributions to the existing IOPS project. This allowed the development work and experimental scripts to be versioned throughout the internship.

The internship also included a meeting with Radita Liem and Sarah Neuwirth (both Johannes Gutenberg University Mainz), and Jay Lofstead (Sandia National Laboratories) as part of the broader collaboration around the IO500 work.

3. Background

This section introduces the elements on which the rest of the report relies. We first present IO500, the benchmark used to measure the performance of HPC storage systems, together with its structure, its scoring rules, and the parameters that can be tuned. We then describe IOPS, the framework used to automate the benchmark executions and to explore the parameter space. The section ends with Bayesian Optimization, the search method investigated in this work.

3.1. IO500

IO500 is a benchmark suite designed to evaluate the I/O performance of storage systems used in High Performance Computing (HPC). It was created to provide a common and reproducible way of measuring storage performance across different systems and configurations [7].

Rather than measuring a single aspect of I/O performance, IO500 combines several benchmarks that represent different types of storage operations. The results can then be used both to characterise a system and to compare it with other systems through the public IO500 ranking [9].

In this work, IO500 is used as the objective to optimise. The following subsections describe how the suite is organised, which benchmarks it uses, how its score is computed, and which execution rules and configuration parameters are relevant to the experiments.

3.1.1. Benchmark structure

IO500 is a benchmark suite rather than a single benchmark program. It combines several benchmarks that evaluate different aspects of I/O performance. The suite provides a common framework for running these benchmarks and collecting their results under the same experimental conditions [7].

The benchmark is organised into a sequence of tests. Each test runs a specific benchmark on the storage system and produces a performance measurement. These measurements are kept separately because they represent different aspects of I/O performance. For example, a benchmark may focus on the amount of data transferred per second, while another may measure the rate of file-system metadata operations.

The benchmarks are executed using a set of parameters that describe how the benchmark should use the available computing and storage resources. These parameters can include the number of compute nodes, the number of processes, and file-system settings. The same benchmark can therefore produce different results depending on the configuration used for the run.

After the individual benchmarks have been executed, their results are combined to produce the overall IO500 score. The score provides a single value that can be used to compare different storage systems or different configurations of the same system. The individual results are also important because they make it possible to identify which aspects of I/O performance contribute to the final score.

IO500 also defines common rules for submitting results to the public ranking. A submission contains not only the measured performance but also information about the system and the configuration used during the experiment [10]. This makes the results more reproducible and allows the conditions of different submissions to be compared.

The suite is used to produce several public lists, including the Production and Research lists. These lists group results according to the conditions under which the systems were evaluated [9].

. The benchmark suite includes `ior-easy`, `ior-hard`, `mdtest-easy`, `mdtest-hard`, and `find` [7, 10]. The IOR tests focus on data transfer performance. They measure how quickly data can be written to and read from the storage system. The `ior-easy` test uses a relatively regular access pattern, while `ior-hard` uses a more demanding pattern. These two tests therefore provide complementary measurements of bandwidth performance.

In contrast, the mdtest benchmarks focus on metadata operations. Instead of measuring the amount of data transferred, they measure operations such as file creation, `stat`, read, and deletion. The `mdtest-easy` and `mdtest-hard` tests use different configurations to exercise the metadata system under varying conditions.

The `find` test measures the performance of searching for files and directories. This operation is different from the operations performed by IOR and mdtest because it involves searching through the file-system namespace.

The benchmark therefore combines tests that stress different parts of the storage system. IOR mainly contributes to the bandwidth measurement, while mdtest and find provide information about metadata and namespace performance. The individual results are kept separately and are then used to compute the final IO500 score [7].

Table 1 summarises the main benchmark tests used in this work.

Table 1: Main IO500 benchmark tests used in this work.

Test	Main tool	Main aspect measured
`ior-easy`	IOR	Sequential I/O bandwidth
`ior-hard`	IOR	More demanding I/O pattern
`mdtest-easy`	mdtest	Metadata performance
`mdtest-hard`	mdtest	Metadata performance
`find`	pfind	File-system search

3.1.2. Scoring and ranking

Each IO500 benchmark produces an individual performance result. These results use different units depending on the type of operation. For example, IOR measures bandwidth, while metadata and search benchmarks measure the rate of file-system operations.

Since these results are not directly comparable, IO500 converts them into scores and combines them to obtain an overall score. The scores of the different benchmarks are combined using a geometric mean. For a set of n benchmark scores $s_1, s_2, \ldots, s_n$, the combined score is computed as

$$S = \left(\prod_{i=1}^{n} s_i \right)^{1/n}. \tag{3.1}$$

The use of the geometric mean gives importance to all benchmarks. A low score in one benchmark

can therefore reduce the overall score even if the other benchmarks perform well. This makes it difficult to obtain a high overall score by optimising only one aspect of I/O performance.

The IO500 benchmarks cover different aspects of storage performance, including bandwidth, metadata operations, and file-system search. Their individual results are combined according to the scoring rules defined by IO500 [10]. The resulting score provides a single value that can be used to compare different systems and configurations.

These scores are then used to build the IO500 ranking. A higher score corresponds to a better position in the ranking, allowing storage systems to be compared under common benchmark and execution rules [9]. The configuration used for the benchmark is therefore an important part of the reported performance.

In this work, the IO500 score is used as the objective value of the optimisation problem. Each configuration evaluated by IOPS produces a score, and the different search strategies aim to find configurations that maximise this value.

3.1.3. Execution rules and stonewall

IO500 defines several rules for running the benchmark. These rules are important because they make it possible to compare results obtained on different systems. In particular, it requires some benchmarks to run for a minimum amount of time.

For an official IO500 submission, the stonewall timer is set to 300 s. This means that the create and write phases must run for at least 300 seconds. If a phase finishes before this time, the run is marked as invalid [10].

The stonewall timer also gives the benchmark enough time to measure the performance of the storage system. Without this minimum time, a short execution could give a result that does not represent the normal performance of the system. This rule is important for this work because one IO500 evaluation can take a significant amount of cluster time. When many configurations have to be tested, running every configuration with the official rules would therefore be very expensive.

3.1.4. Configuration and tuning

The performance obtained by IO500 depends not only on the storage hardware, but also on how the benchmark uses the available resources. Several parameters can therefore be changed between two executions. At the compute level, the configuration can include the number of compute nodes and the number of processes used by the benchmark. The placement of processes can also affect the communication between the compute nodes and the storage system.

The file system configuration provides additional parameters. For example, the number of storage targets used by a file and the way data is distributed across these targets can affect I/O performance. Other parameters, such as transfer size and block size, can also influence the measured performance.

These parameters are not independent. A configuration that performs well with a small number of processes may not provide the same performance when more processes or nodes are used. The result is a multi-dimensional configuration space in which each combination of parameter values corresponds to a possible IO500 execution.

3.2. IOPS

The previous section showed that tuning IO500 can require a large number of benchmark evaluations. Each evaluation can take a significant amount of cluster time, making manual testing and exhaustive search difficult in practice. To automate this process, we use IOPS [13]. IOPS is a framework developed within the TADaaM team to run parameter studies on HPC systems. It can generate benchmark runs, submit them to the cluster, collect their results, and use these results to guide a search.

This makes IOPS suitable for the IO500 tuning problem. Instead of manually changing the parameters and launching each benchmark, the parameters and their possible values can be described in an IOPS configuration file. The search strategy can then select which configuration should be evaluated next.

In this work, IOPS is used as the common framework for comparing different search strategies. This allows exhaustive search, random search, and Bayesian Optimization to use the same benchmark and experimental environment. The main difference between the experiments is therefore the way in which the next configuration is selected.

3.2.1. IOPS workflow

An IOPS experiment starts with a configuration file that describes the parameters to be tested and the benchmark to be executed. From this configuration, IOPS generates the different parameter combinations that will be evaluated.

For each evaluation, IOPS prepares the benchmark command with the selected parameters and submits the job to the HPC cluster. The cluster scheduler then executes the benchmark on the allocated resources. Once the execution is finished, IOPS collects the output and extracts the performance result. The result is associated with the parameters used for the corresponding run. It can then be used by the search strategy to select the next configuration to evaluate. The process is repeated until the search budget is reached or all configurations have been evaluated.

IOPS also provides a cache for the results. When a configuration has already been evaluated, its result can be retrieved from the cache instead of running the benchmark again. This is particularly useful for IO500, where a single valid evaluation can require a significant amount of cluster time. The general workflow used in this internship is summarised in Figure 2.

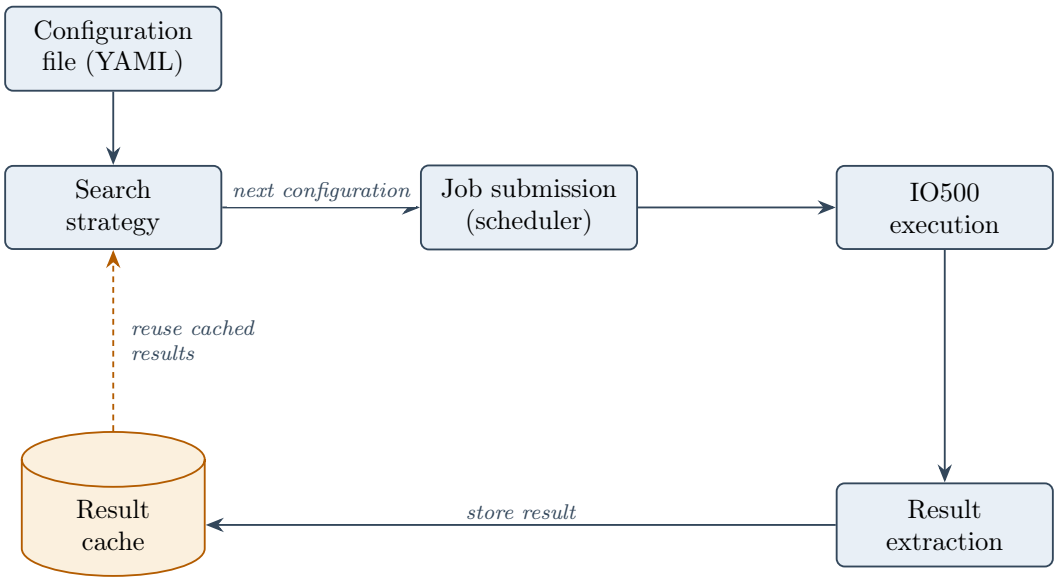


Figure 2: Workflow of an IO500 experiment in IOPS. The search strategy selects the next configuration, IOPS submits the corresponding benchmark job to the cluster scheduler, and the extracted result is stored in the cache. Configurations that have already been evaluated are served from the cache instead of being run again.

This workflow allows the benchmark execution and the search strategy to remain separate. The same IO500 experiment can therefore be used with different search strategies without changing the rest of the experimental setup.

3.2.2. Parameter configuration

IOPS uses a YAML configuration file to describe an experiment. The file contains the parameters to explore, the values that can be used for each parameter, and the commands needed to run the benchmark. This makes the experimental setup explicit and allows the same study to be reproduced later.

For the IO500 experiments, the configuration file describes the parameters that can affect the benchmark performance. For example, the number of compute nodes, the number of processes per node, and storage-related parameters can be included in the study. Each possible value is specified in the YAML file, and IOPS uses these values to generate the configurations to evaluate.

A simplified example of the configuration used for this work is shown below:

```yaml
benchmark:
  name: io500

parameters:
  nodes:
    values: [1, 2, 4]
  processes_per_node:
    values: [1, 2, 4, 8]
```

The YAML configuration defines the benchmark and the parameter space to explore, while the search strategy determines how configurations are selected. Therefore, the same parameter space can be explored using different strategies available in IOPS, such as exhaustive search, random search, or Bayesian Optimization.

During the internship, an example IO500 YAML configuration was also created as part of the work. This configuration provides a reproducible description of the experiment and can be reused to run IO500 through IOPS.

3.2.3. Search strategies

IOPS supports several search strategies for exploring a parameter space. The search strategy determines which configuration is selected at each step of an experiment. The simplest approach is exhaustive search. It evaluates all possible parameter combinations and provides the best result within the defined search space. However, the number of configurations can quickly become too large.

Random search selects configurations randomly from the parameter space. It does not use the results of previous evaluations to select the next configuration. It is therefore simple to use and provides a useful baseline for comparison. Moreover, IOPS can associate a budget with random search, expressed as a number of tests or a number of core hours. This makes the strategy suitable when the objective is to explore a large parameter space without relying on a guided method.

Bayesian Optimization (BO) is the main search strategy investigated in this work. It uses the results of previous evaluations to guide the selection of new configurations. However, BO is not a single fixed method. Its behaviour depends on several choices, such as the surrogate model, the acquisition function, the exploration parameters, the number of initial evaluations, and the total search budget.

These parameters can have a significant effect on the quality of the search. For example, the surrogate model determines how the results of previous evaluations are represented, while the acquisition function controls how new configurations are selected. The exploration parameters also influence the balance between testing promising configurations and exploring less tested areas of the parameter space.

The search component of IOPS was also extended during the internship with additional optimization methods. The Tree-structured Parzen Estimator (TPE) was implemented as an additional search strategy. Iterated Racing (irace) and Covariance Matrix Adaptation Evolution Strategy (CMA-ES) were also considered as possible extensions. All these strategies can use the same IOPS experiment and parameter space.

3.2.4. Result management and caching

IOPS stores the result of each experiment together with the parameters used for that experiment. This allows the results to be kept and reused after the benchmark execution. The results are stored in an SQLite database. Each entry associates a parameter configuration with its corresponding benchmark result. The database therefore provides a persistent record of the experiments performed during the study.

The cache also makes it possible to replay an experiment without using the cluster. For example,

IOPS can use the previously stored results in cache-only mode instead of submitting new benchmark jobs. This was used to compare search strategies offline using the same set of IO500 measurements.

3.3. Bayesian Optimization

BO is a search method designed for problems where evaluating all possible configurations is too expensive. This makes it suitable for the IO500 tuning problem, since each configuration requires running the benchmark on an HPC system and can therefore consume a significant amount of cluster time. In addition, the number of parameter combinations is very large. The main idea of BO is to use the results of previous evaluations to decide which configuration should be evaluated next. In other words, it explores the parameter space selectively rather than uniformly.

Let the configuration of a benchmark be represented by a vector of parameters x, and let $f(x)$ be the performance obtained by running IO500 with this configuration. The objective is to find a configuration that maximises $f(x)$. In a large search space, evaluating every possible configuration may be too expensive. BO instead tries to find a good configuration with a limited number of evaluations.

BO starts with a small number of initial configurations. These configurations are evaluated using the real objective function, in our case the IO500 benchmark. The obtained results form the initial set of observations. BO then builds a model, called a *surrogate model*, from these observations. This model provides an approximation of the relationship between the configuration parameters and the expected performance. The surrogate model is used because evaluating the real objective function is expensive, while evaluating the model is much cheaper. The model can therefore be used to estimate which parts of the search space are promising before running another IO500 experiment.

After building the surrogate model, BO uses an *acquisition function* to select the next configuration to evaluate. The acquisition function uses the information provided by the model to decide where the next evaluation should be performed. It can favour configurations that are expected to provide a high score, but it can also favour configurations where the model is uncertain.

This creates a balance between *exploitation* and *exploration*. Exploitation means testing configurations that are expected to perform well according to the current model. Exploration means testing less certain configurations in order to obtain new information about the search space. Finding a good balance between these two behaviours is important because focusing only on exploitation can cause the search to remain in a local region, while excessive exploration can waste evaluations on configurations that are unlikely to perform well.

The process is then repeated. The selected configuration is evaluated with IO500, its measured score is added to the observations, and the surrogate model is updated. A new configuration is then selected using the updated model. In this way, each expensive evaluation provides information that can be used to improve the following choices.

The general workflow of BO is illustrated in Figure 3.

An important point is that BO is not a single fixed algorithm. Different surrogate models and acquisition functions can be used, and their parameters can change the behaviour of the search. In this work, we consider Gaussian Processes (GP), Random Forests (RF), and Extra-Trees (ET) as surrogate models. We also consider Expected Improvement (EI), Probability of Improvement (PI),

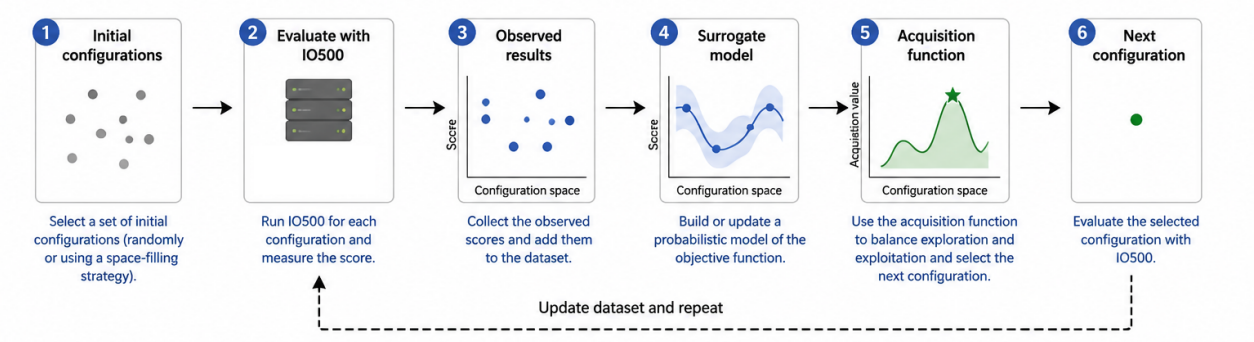


Figure 3: General workflow of Bayesian Optimization for IO500 tuning.

and Lower Confidence Bound (LCB) as acquisition functions.

Other parameters also affect the search, such as the number of initial evaluations and the parameters controlling exploration. The total number of evaluations, or search budget, is another important factor because it determines how many runs can be performed.

For these reasons, the objective of the study is not simply to determine whether BO works in this context, but also to identify which combination of BO choices is most effective for the IO500 tuning problem. The different BO configurations are evaluated under the same experimental conditions and with a limited number of evaluations. Their performance can then be compared with the exhaustive and random search strategies described previously.

4. Methodology

This section describes how the study was carried out. We first define the parameter space explored in the experiments, which covers the IO500 benchmarks, the compute environment, and the storage system. We then present the exhaustive search used to evaluate every configuration in this space, and the reference dataset built from its results. This dataset provides the ground truth against which the search strategies are compared. The last part of the section describes these strategies, namely random search and the different Bayesian Optimization configurations.

4.1. Parameter space

The first step of the study was to identify the parameters that could affect the performance of an IO500 execution. The configuration of IO500 contains parameters related to the I/O benchmarks, but the performance also depends on the execution environment, including the compute nodes and the parallel file system. We therefore considered parameters from three main aspects: the IO500 benchmarks, the compute environment, and the storage system.

Table 2 summarises the parameters identified in these different parts of the system. The role of each group of parameters is explained in more detail in the following paragraphs.

Table 2: Parameters considered in the IO500 tuning study.

Category	Parameter	Description
IOR	`transferSize`	Size of each I/O operation
IOR	`blockSize`	Data handled by each process
IOR	`filePerProc`	One file per process
IOR	`segmentCount`	Number of file segments
MDTEST	`n`	Number of files per process
MDTEST	`files-per-dir`	Number of files per directory
FIND	`nproc`	Number of processes
FIND	`pfind-queue-length`	Size of the work queue
FIND	`pfind-parallelize...`	Parallel directory access
Machine	`nodes`	Number of compute nodes
Machine	`processes-per-node`	Number of processes per node
BeeGFS	`ost_count`	Number of storage targets used

IOR parameters. The IOR parameters mainly control the way data is transferred during the bandwidth benchmarks. The `transferSize` determines the amount of data transferred by each I/O operation. Changing this value can modify the number of I/O requests sent to the storage system and therefore affect the measured bandwidth.

The `blockSize` determines the amount of data handled by each process. Together, `transferSize` and `blockSize` define an important part of the I/O access pattern generated by the benchmark.

The `filePerProc` parameter determines whether each process works on its own file or whether several processes access a shared file. This choice can strongly change the level of contention and the way the parallel file system handles the benchmark.

Finally, `segmentCount` controls the number of segments used by the IOR benchmark. Increasing the number of segments changes the total amount of data processed and the sequence of I/O operations performed by each process.

MDTEST parameters. The MDTEST parameters control the metadata benchmark generated by IO500. The parameter `n` determines the number of files handled by each process. A larger value therefore generates more file-system metadata operations.

The `files-per-dir` parameter controls how these files are distributed between directories. This can affect performance because placing many files in the same directory does not generate the same metadata access pattern as distributing them across several directories.

Find parameters. The FIND benchmark evaluates the performance of file discovery operations. The `nproc` parameter determines the number of processes involved in the search and therefore controls the level of parallelism.

The `pfind-queue-length` parameter controls the amount of work that can be kept in the search queue. The `pfind-parallelize-single-dir-access-using-hashing` parameter allows accesses to a directory to be distributed between several processes. These parameters influence how the search work is divided and executed in parallel.

Compute environment parameters. The number of compute nodes and the number of processes per node also affect IO500 performance. The `nodes` parameter determines how many `bora` nodes participate in an execution, while `processes-per-node` determines how many benchmark processes are started on each node.

Together, these two parameters determine the total number of processes accessing the storage system. Increasing this number can provide more I/O parallelism, but it can also increase contention when many processes access the storage system simultaneously.

Parallel file system parameter. The experiments use BeeGFS as the parallel file system. The `ost_count` parameter controls the number of storage targets used to distribute file data.

Using a larger number of targets can distribute I/O operations across more storage resources and potentially increase the available bandwidth. However, the best value depends on the benchmark and on the number of processes accessing the storage system. The effect of this parameter must therefore be considered together with the IOR and compute parameters.

From the parameters to the search space. These parameters do not act independently. For example, increasing the number of processes can change the benefit obtained from using more storage targets, while the effect of a given `transferSize` can depend on the level of parallelism used during the execution.

A configuration is therefore defined by a combination of values from the different parameter categories. The set of all possible combinations forms the parameter space of the study. Exploring this space makes it possible to study the interactions between parameters and to identify configurations that provide high IO500 scores.

However, the number of possible combinations quickly becomes large. This makes a complete evaluation expensive when each configuration requires an IO500 execution. The following sections describe how this parameter space was explored and how a reduced-cost exhaustive campaign was used as a reference for the evaluation of the search strategies.

4.2. Exhaustive search

The first approach used in this study was exhaustive search. The idea was to test every possible configuration in the parameter space in order to identify the best IO500 configuration.

The goal of this work is not to use exhaustive search as a practical tuning method. Testing all configurations is too expensive for this purpose. Instead, exhaustive search is used to create a reference dataset. This reference contains the results of all tested configurations and can later be used to compare Random Search and the different Bayesian Optimization methods.

Cost of a valid exhaustive search. The most direct way to build this reference would be to run every configuration using the official IO500 settings. However, this would take too much time.

Before starting the experiments, IOPS was used to estimate the execution time required for the complete exhaustive search. With valid IO500 executions, the estimated duration was approximately 161 days.

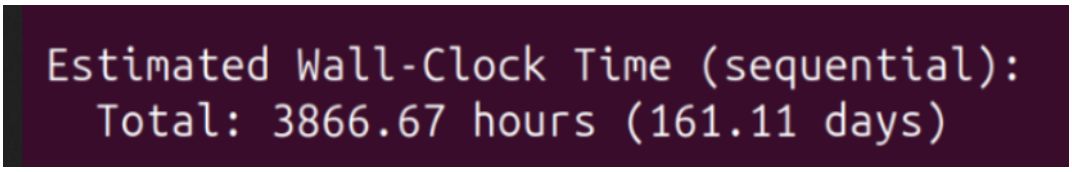


Figure 4: Execution time estimated by IOPS for an exhaustive search using valid IO500 executions.

This duration was close to the complete length of the internship, which lasted five months. In addition, the experiments were executed on the PlaFRIM `bora` nodes. These nodes are shared with other users. Running the complete exhaustive search with valid IO500 executions would therefore require using and reserving compute resources for a very long period. This was not practical on a shared research platform.

Reduced-cost IO500 executions. For this reason, a faster way to evaluate the configurations was needed. The idea was to reduce the cost of each IO500 execution.

Two main changes were made. First, the stonewall time was reduced from 300 seconds to 10 seconds. Second, the size-related parameters used in the benchmark were divided by 10. These changes reduced the amount of work performed during each execution and made the tests much faster.

With these reduced settings, some IO500 phases do not satisfy the official execution requirements. The final IO500 result is therefore marked as `INVALID`. These results cannot be used as official IO500 submissions.

However, the purpose of these short tests is not to produce an official IO500 score. Their purpose is to provide a cheaper way to compare configurations under the same experimental conditions.

The main question was therefore whether these short executions could still give useful information about the quality of a configuration. In particular, we wanted to know whether configurations that perform well with the reduced settings also tend to perform well with valid IO500 settings.

Comparison between valid and invalid executions. To study this question, several configurations were executed using both the reduced and the valid IO500 settings. Their scores and bandwidth results were then compared.

Figure 5 shows this comparison.

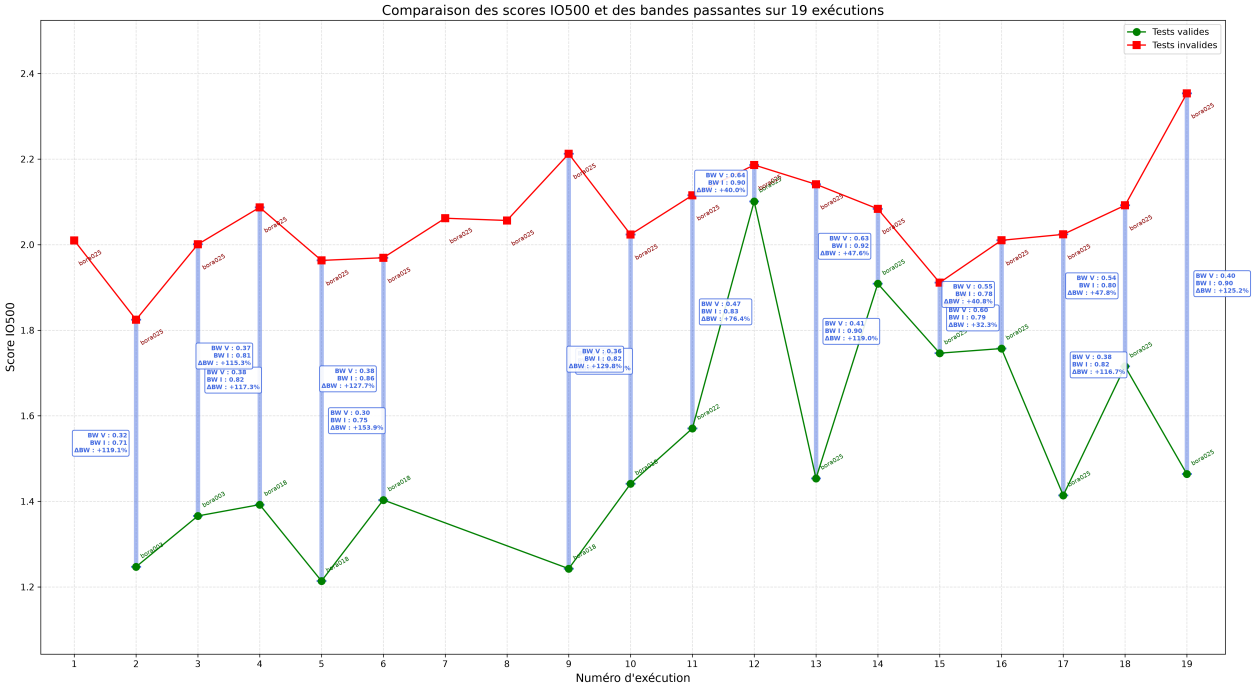


Figure 5: Comparison of IO500 scores obtained with valid and invalid executions for selected configurations.

The comparison shows that the scores obtained with short executions are different from the valid IO500 scores. The difference is also not the same for every configuration. Therefore, a simple correction value cannot be used to convert an invalid score into a valid score.

However, reproducing the exact valid score is not the objective of the reduced-cost tests. What matters for this study is whether they provide enough information to distinguish good configurations from poor ones. If a good configuration can still be identified with a short execution, the complete parameter space can be explored at a much lower cost.

For this reason, the reduced settings were used for the exhaustive search. All configurations were evaluated under the same conditions, which makes their results comparable within this study.

Repetitions. I/O performance can vary between executions, even when the same configuration is used. Temporary activity on the compute or storage system can affect the measured performance.

For this reason, each configuration was executed three times. These repetitions reduce the effect of temporary variations and provide more information about the stability of each configuration.

The exhaustive search contained 4,830 possible configurations. With three repetitions for each configuration, the complete experiment therefore represented:

$$4\,830 \times 3 = 14\,490$$

IO500 executions.

All repetitions used the same reduced-cost settings, including the 10-second stonewall time and the reduced size-related parameters.

Execution with IOPS. The experiments were defined using a YAML configuration file interpreted by IOPS. This file contains the benchmark settings, the parameter values, the execution method, the SLURM configuration, and the way the results are collected.

The parameters that have to be explored are defined in the `vars` section. For example, the number of nodes, the number of processes per node, the transfer size, the number of storage targets, `file_per_proc`, and `unique_dir` can each take several values.

Some parameters are not explored directly but are computed from other parameters. For example, the total number of MPI tasks is calculated as:

$$\texttt{ntasks} = \texttt{nodes} \times \texttt{processes_per_node}.$$

The block size is also computed from the total data size and the number of tasks:

$$\texttt{block_size_mb} = \frac{\texttt{total_size_gb} \times 1024}{\texttt{ntasks}}.$$

IOPS can also apply constraints before running a configuration. In this study, the block size had to be a multiple of the transfer size. Invalid combinations were skipped automatically:

```
constraints:
  - name: "block_transfer_alignment"
    rule: "block_size_mb % transfer_size_mb == 0"
    violation_policy: "skip"
```

This is important because not every mathematical combination of the parameter values represents a valid IO500 configuration.

The YAML file is also used to generate the IO500 configuration file for each execution. IOPS replaces the parameter values in the template before submitting the job. For example, the transfer size, block size, file-per-process option, and storage directory are generated from the values selected for the current configuration.

The execution itself is submitted to SLURM. The job script defines the number of nodes, the total number of MPI tasks, and the number of processes per node. The jobs are executed on the PlaFRIM `bora` nodes using an exclusive allocation.

Each configuration is executed three times, as defined by:

```
repetitions: 3
```

The search strategy is also selected directly in the YAML file. This means that the same benchmark definition can be used with Exhaustive Search, Random Search, or Bayesian Optimization by changing the search method while keeping the rest of the experimental setup unchanged.

Result collection. After each execution, IOPS parses the IO500 output and extracts the main performance metrics. In this study, the parser collects the overall IO500 score, the bandwidth result, and the metadata result:

```
metrics:
  - name: SCORE
  - name: BW
  - name: MD
```

IOPS also stores information about the configuration and the execution. At the end of the experiment, these values are written to a CSV file, with one row for each execution.

The result file contains several groups of columns. The `vars.*` columns describe the configuration used for the execution. They include, for example, `vars.nodes`, `vars.processes_per_node`, `vars.transfer_size_mb`, `vars.ost_count`, `vars.file_per_proc`, and `vars.unique_dir`.

The `execution.*` columns identify the execution and its repetition. The `metadata.*` columns contain information about the SLURM job and the machine on which it was executed. Finally, the `metrics.*` columns contain the IO500 results, in particular `metrics.SCORE`, `metrics.BW`, and `metrics.MD`.

This CSV file therefore gives, for every execution, both the parameter values that were used and the performance obtained with these values. It is the main source of data used to build the reference dataset for the rest of the study.

4.3. Ground-truth construction

The exhaustive search produced a large number of IO500 results for the parameter space used in this study. These results were then used to build a reference dataset for comparing the different search methods.

For each execution, IOPS stores the parameter values and the corresponding IO500 results. The main information used in this study includes the configuration parameters, the repetition number, and the three main metrics: `SCORE`, `BW`, and `MD`.

Each configuration was executed three times. This provides several measurements for the same configuration and helps reduce the effect of temporary performance variations. It also gives a more reliable view of the performance of each configuration.

The resulting dataset is used as the ground truth in this study. Here, ground truth does not mean that the results are official valid IO500 results. As explained in the previous section, the exhaustive search was performed with shorter IO500 executions, which are marked as `INVALID`. The ground truth is therefore a reference for the reduced-cost setup used in this work.

The main advantage of this dataset is that the results of all tested configurations are already known. It is therefore possible to identify the best configuration in the reference dataset.

This reference is then used to evaluate the search methods. For example, if Bayesian Optimization is allowed to test only 40 configurations, the best configuration it finds can be compared with the

best configuration in the complete reference dataset. This shows how close the method gets to the best known result with only a small number of evaluations.

Caching and offline evaluation. The results were also stored in the IOPS cache file `io500.db`. The cache links each configuration to the result already measured for it.

This is useful when comparing Random Search and Bayesian Optimization. When a search method selects a configuration, there is no need to run IO500 again. IOPS can directly read the result from the cache.

The `-cache-only` option was used for this purpose. With this option, IOPS only uses results already stored in the cache and does not submit new jobs to the cluster.

This makes it possible to repeat the search experiments many times without using additional `bora` resources. It also ensures that all search methods are compared using the same IO500 measurements.

The reference dataset and the cache are therefore used for the offline comparison of the search methods described in the next section.

4.4. Search strategies

4.4.1. Random Search

Random Search was used as a simple search method in this study. Instead of testing every possible configuration, it selects a subset of the parameter space at random.

In IOPS, Random Search is enabled directly in the YAML configuration by setting:

```yaml
search_method: "random"
```

The amount of the parameter space to explore can also be defined in the Random Search configuration. For example:

```yaml
random_config:
  percentage: 0.05
```

In this example, Random Search selects 5% of the available configurations. Other percentages can be used depending on the number of configurations that should be evaluated.

The selected configurations come from the same parameter space defined for the study. The same constraints are applied, so configurations that do not satisfy the experimental rules are not considered.

Random Search does not use the results of previous executions to choose new configurations. The configurations are selected randomly from the available search space.

This makes Random Search simple to configure and allows only a part of the complete parameter space to be evaluated.

4.4.2. Bayesian Optimization

Bayesian Optimization was the main guided search method studied during the internship. Unlike Random Search, it uses the results of previous evaluations to decide which configuration should be tested next.

In IOPS, Bayesian Optimization is enabled by setting:

```
search_method: "bayesian"
```

The objective of the search is also defined in the configuration. In this study, the IO500 `SCORE` is the value to maximise:

```
bayesian_config:
  objective_metric: "SCORE"
  objective: "maximize"
```

Bayesian Optimization starts by evaluating a small number of initial configurations. These first results are used to build a model of the relation between the parameter values and the IO500 score. The model is then used to select the next configuration. After this configuration is evaluated, the new result is added to the previous observations and the model is updated. This process continues until the evaluation budget is reached.

Bayesian Optimization is not a single fixed method. Several choices can be made for the model and for the strategy used to select the next configuration. In IOPS, these choices are controlled by the `bayesian_config` section.

Surrogate models. The surrogate model estimates the IO500 score of configurations that have not yet been evaluated. Three surrogate models were considered in this study:

- `GP`: Gaussian Process;
- `RF`: Random Forest;
- `ET`: Extra Trees.

All three models learn from the configurations already evaluated, but they represent the relation between parameters and performance in different ways. The choice of surrogate model can therefore change the configurations proposed during the search.

The model is selected with the `base_estimator` option. For example:

```
base_estimator: "RF"
```

selects a Random Forest as the surrogate model.

Acquisition functions. Once the surrogate model has been built, an acquisition function is used to choose the next configuration to evaluate. Three acquisition functions were considered:

- `EI`: Expected Improvement;

- `PI`: Probability of Improvement;
- `LCB`: Lower Confidence Bound.

EI considers the expected improvement over the best result found so far. PI considers the probability that a configuration will improve the current best result. LCB uses the prediction of the model together with its uncertainty to decide where to search next.

These functions provide different ways to balance two important behaviours. The first is exploitation, which means testing configurations that are expected to perform well. The second is exploration, which means testing less-known parts of the parameter space.

The acquisition function is selected with, for example:

```
acquisition_func: "EI"
```

Exploration parameters. The balance between exploration and exploitation can also be controlled by additional parameters.

For EI and PI, the parameter `xi` controls the amount of exploration. Different values can therefore make the search more focused on the current best region or encourage it to explore other parts of the parameter space.

For LCB, this role is controlled by `kappa`. A larger value gives more importance to uncertainty and therefore encourages more exploration.

For example, a BO configuration can use:

```
xi: 0.01
kappa: 1.96
```

Only the parameter related to the selected acquisition function has an effect on the search.

Initial points and evaluation budget. The parameter `n_initial_points` defines how many configurations are evaluated before the surrogate model starts guiding the search. These initial configurations provide the first observations needed to build the model.

The parameter `n_iterations` controls the number of evaluations performed by the optimisation. It therefore represents an important part of the search budget. A larger budget gives Bayesian Optimization more opportunities to learn about the parameter space, but it also requires more evaluations.

For example:

```
n_initial_points: 5
n_iterations: 10
```

starts the optimisation with five initial points and uses an evaluation budget defined by ten iterations.

Example of a Bayesian configuration. The following configuration is one example of a Bayesian Optimization setup used with IOPS:

```yaml
search_method: "bayesian"

bayesian_config:
  objective_metric: "SCORE"
  objective: "maximize"
  n_initial_points: 5
  n_iterations: 10
  acquisition_func: "EI"
  base_estimator: "RF"
  xi: 0.01
  kappa: 1.96
  fallback_to_exhaustive: false
```

This example corresponds to a Random Forest surrogate combined with the Expected Improvement acquisition function. It uses five initial points and `xi = 0.01`. This is only one possible BO configuration.

In this study, several combinations of surrogate models, acquisition functions, exploration parameters, initial points, and evaluation budgets were tested. This is important because changing one of these options can change the behaviour of Bayesian Optimization. The goal was therefore not only to test whether Bayesian Optimization works for IO500, but also to determine which BO configuration is the most effective for this problem.

Selection of the next configuration. IOPS implements the Bayesian search as a sequential process. At each iteration, the optimiser suggests a new point based on the results already available. IOPS then maps this suggestion to a valid configuration from the parameter space. The result of this configuration is returned to the optimiser and used to update the model.

The process can therefore be summarised as:

$$\text{suggest} \rightarrow \text{select a valid configuration} \rightarrow \text{evaluate} \rightarrow \text{update the model.}$$

Because each new suggestion depends on the previous results, Bayesian Optimization is sequential: the result of one evaluation is used before selecting the next configuration.

5. Results

5.1. Characterisation of the response surface

- Distribution of the SCORE over the whole measured grid: how many configurations are near-optimal, and how heavy is the tail. (`score_distribution.csv`, `fig_score_distribution.png`)
- 2D projections of the response surface (`fig_response_surface_2d.png`): show that the landscape is neither convex nor separable.

- **Takeaway to state explicitly**: the optimum is isolated and the surface has interactions, which is why coordinate-wise greedy tuning is structurally at a disadvantage.

5.2. Which Bayesian Optimization setup wins?

- Ranking of the 83 BO configurations (`best_setup_ranking.png`).
- **Main finding: the surrogate matters most – the top-5 setups are all Gaussian Process.** Acquisition (LCB vs EI) is close to a tie at the top.
- Sensitivity analysis, one knob at a time: rank the influence of estimator, acquisition, ξ/κ and n_{initial}.
- Practical recommendation for a practitioner who will run this once: state a single default tuple (surrogate, acquisition, exploration, n_{initial}) and justify it.

5.3. Why random search is not enough

- Analytic argument from the measured score distribution: reaching SCORE ≥ 15 with 90 % probability by chance requires $\approx$ 1,081 tests.
- **Takeaway**: the near-optimal tail is effectively unreachable by luck; random search is a good baseline for *average* quality and a bad strategy for *peak* quality.
- Nuance to keep (it is honest and it strengthens the report): at very low budgets, random is competitive with, and sometimes better than, the tree-based surrogates.

5.4. Automated search versus a human specialist

- Protocol recap and the specialist's submitted candidates.
- **Result: the expert's best informed pick reached $\approx$ 95 % of the optimum; BO with 40 evaluations reached $\approx$ 98 %** (`specialist_vs_bo.png`).
- Discuss fairly: the expert answered in minutes and without access to the machine's response surface; the interesting claim is not "the machine beats the human" but "a modest automated budget matches or exceeds expert intuition, reproducibly".
- Limitation to state: only one specialist has submitted so far, so this is an illustration, not a statistically meaningful comparison.

5.5. Validation with valid IO500 runs

6. Conclusion and perspectives

6.1. Summary

- Problem: tuning IO500 properly is intractable because the validity rules make a single measurement cost $\approx$ 30 min.
- Method: measure the whole grid once with deliberately *invalid* but rank-preserving short runs, cache it, and replay every search strategy offline at zero cluster cost.

- Result: the exhaustive campaign cost $\approx 183\,\mathrm{h}$ instead of more than a year, and a well-chosen Bayesian Optimization setup reaches $\geq 98\,\%$ of the optimum in a few dozen evaluations, one order of magnitude cheaper than exhaustive search and above what random search or a human expert reached.
- Contribution to the tooling: the methodology, the configurations and the analysis scripts are packaged with IOPS and are reusable on another machine.

6.2. Perspectives

- Complete the **valid cluster validation** of the best configurations and report official-grade IO500 scores.
- Reproduce the study on **IRENE** (TGCC), a larger and more complex system, to test generalisation.
- Extend the **10-node challenge** study.
- Collect submissions from **more specialists** to make the human baseline statistically meaningful.
- Implement and compare **additional search techniques** against the existing cache: genetic algorithms, simulated annealing, greedy/coordinate search (to reproduce the CLAIX23 baseline directly on our data).
- Feed the diagnostic findings back into the IOPS `BayesianPlanner` (surrogate defaults, handling of discrete dimensions, duplicate-proposal policy).
- Longer term: transfer learning across machines – reuse the response surface measured on one system to warm-start the search on another.

Bibliography

References

[1] Michael A. Heroux, Douglas W. Doerfler, Paul S. Crozier, James M. Willenbring, H. Carter Edwards, Alan Williams, Mahesh Rajan, Eric R. Keiter, Heidi K. Thornquist, and Robert W. Numrich. Improving performance via mini-applications. Technical Report SAND2009-5574, Sandia National Laboratories, 2009. VERIFY: author list and report number.

[2] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015. VERIFY: article number / pages.

[3] Inria. Inria centre at the university of bordeaux. `https://www.inria.fr/en/inria-centre-university-bordeaux`. Accessed: 2026-08-24.

[4] Inria. Plafrim – cluster of inria of bordeaux. `https://plafrim-users.gitlabpages.inria.fr/doc/`. Accessed: 2026-08-24.

[5] Inria. The project-teams – distributed and high performance computing. `https://www.inria.fr/en/list-of-project-teams`. Accessed: 2026-08-24.

[6] Inria. Tadaam – topology-aware system-scale data management for high-performance computing. `https://www.inria.fr/en/tadaam`. Accessed: 2026-08-24.

[7] IO500 Foundation. Io500 – about. `https://io500.org/about`. Accessed: 2026-08-27.

[8] IO500 Foundation. Io500 – running. `https://io500.org/running`. Accessed: 2026-08-27.

[9] IO500 Foundation. The IO500 list. `https://io500.org/`, 2026. Accessed: September 8, 2026.

[10] IO500 Foundation. IO500 submission rules. `https://io500.org/rules/submission`, 2026. Accessed: September 8, 2026.

[11] Frank Schmuck and Roger Haskin. GPFS: A shared-disk file system for large computing clusters. In *Proceedings of the 1st USENIX Conference on File and Storage Technologies (FAST)*, pages 231–244, 2002. VERIFY: page numbers.

[12] Philip Schwan. Lustre: Building a file system for 1000-node clusters. In *Proceedings of the 2003 Linux Symposium*, 2003. VERIFY: page numbers.

[13] TADaaM, Inria. IOPS: Integrated orchestration for parametric studies. `https://iops-benchmark.com`, 2026. Source repository: `https://gitlab.inria.fr/hpc_io/iops`. Accessed: September 8, 2026.

[14] The IOR contributors. IOR and mdtest: Parallel filesystem I/O benchmarks. `https://github.com/hpc/ior`, 2026. Accessed: September 8, 2026.

[15] The IOR contributors. mdtest: Metadata benchmark for parallel file systems. `https://github.com/hpc/ior`, 2026. Distributed with the IOR suite. Accessed: September 8, 2026.

[16] The scikit-optimize contributors. scikit-optimize: Sequential model-based optimization in Python. https://scikit-optimize.github.io/, 2026. Version TO-COMPLETE.

[17] ThinkParQ GmbH. BeeGFS: The parallel cluster file system. https://www.beegfs.io/, 2026. Accessed: September 8, 2026.

[18] TO-COMPLETE. To-complete: 2022 I/O performance study. https://inria.hal.science/hal-03753813/, 2022.

[19] TO-COMPLETE. To-complete: IO500 tuning on CLAIX23. In *IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*, 2025.

[20] TO-COMPLETE. To-complete: IO500 benchmark reference paper, 2026.

[21] TOP500.org. The TOP500 list. https://www.top500.org/, 2026. Accessed: September 8, 2026.